# useState

## What is `useState` ?

`useState` is a React hook that lets a component **remember a value between renders**. Updating that value triggers a re-render with the new value.

```
const [state, setState] = useState(initialValue);
```

| Parameter | Description |
|---|---|
| `initialValue` | The starting value. Used only on the first render. |
| `state` | The current value for this render. |
| `setState` | A function to update the value and trigger a re-render. |

## When to use `useState` vs `useReducer`

| Situation | Use |
|---|---|
| Simple, independent values | `useState` |
| A single boolean, number, or string | `useState` |
| A controlled input value | `useState` |
| Multiple related values in one object | `useReducer` |
| Multiple actions affect the same state | `useReducer` |
| Logic is complex enough to benefit from tests | `useReducer` |

## Core Concepts

### 1. Initial value

The value used on the **first render only**. On every later render, `useState` returns the current value, not this one.

```
const [count, setCount] = useState(0);        // number
const [name, setName]   = useState("");       // string
const [open, setOpen]   = useState(false);    // boolean
const [items, setItems] = useState<number[]>([]); // typed array
```

> If computing the initial value is expensive, pass a **function**: `useState(() => heavyCompute())`. React will call it only on the first render.

## 2. The setter — direct update

Use when the next value **does not depend** on the current one.

```
setCount(0);
setName("Alice");
setOpen(true);
```

## 3. The setter — functional update

Use when the next value **depends on the previous one**. The setter passes you the latest value.

```
setCount((c) => c + 1);
setItems((prev) => [...prev, newItem]);
setItems((prev) => prev.filter((x) => x !== target));
```

> Without the functional form, multiple updates in the same event can lose intermediate values because they all read the same stale `count`.

## 4. State is a snapshot

Inside one render, `state` is a **constant**. Calling `setState` doesn't change it here — it schedules a new render where the value will be updated.

```
function handleClick() {
  setCount(count + 1);
  console.log(count);   // still the old value in THIS render
}
```

## 5. State must be treated as immutable

Never mutate arrays or objects held in state. Always produce a new value.

```
// ❌ mutation — React sees the same reference and may skip re-render
items.push(newItem);
setItems(items);

// ✅ new array
setItems([...items, newItem]);

// ✅ new object
setUser({ ...user, name: "Bob" });
```

---

## Full Example — Counter (number)

```
import { useState } from "react";

export default function Counter() {
  const [count, setCount] = useState<number>(0);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount((c) => c + 1)}>+</button>
      <button onClick={() => setCount((c) => c - 1)}>-</button>
      <button onClick={() => setCount(0)}>Reset</button>
    </div>
  );
}
```

> `+` and `-` use the functional form because the next count depends on the previous one. `Reset` uses the direct form because `0` doesn't depend on the current count.

---

## Full Example — Toggle (boolean)

```
import { useState } from "react";

export default function Toggle() {
  const [on, setOn] = useState<boolean>(false);
```

```
  return (
    <button onClick={() => setOn((prev) => !prev)}>
      {on ? "ON" : "OFF"}
    </button>
  );
}
```

> `setOn((prev) => !prev)` flips the current value — a textbook case for the functional updater.

---

## Full Example — Controlled Text Input (string)

```
import { useState } from "react";

export default function NameInput() {
  const [name, setName] = useState<string>("");

  return (
    <div>
      <input
        value={name}
        onChange={(e) => setName(e.target.value)}
        placeholder="Type your name"
      />
      <p>Hello, {name || "stranger"}!</p>
    </div>
  );
}
```

> The `<input>` reads from state ( `value={name}` ) and writes to it ( `onChange` ). This is called a **controlled input**.

---

## Full Example — Object State (form fields)

```
import { useState } from "react";

type Form = { name: string; email: string };
```

```
export default function SignupForm() {
  const [form, setForm] = useState<Form>({ name: "", email: "" });

  function updateField(field: keyof Form, value: string) {
    setForm((prev) => ({ ...prev, [field]: value }));
  }

  return (
    <form>
      <input
        value={form.name}
        onChange={(e) => updateField("name", e.target.value)}
        placeholder="Name"
      />
      <input
        value={form.email}
        onChange={(e) => updateField("email", e.target.value)}
        placeholder="Email"
      />
    </form>
  );
}
```

> `{ ...prev, [field]: value }` spreads the previous object, then overrides one key. This keeps state **immutable**.

---

## Full Example — Array State (todo list)

```
import { useState } from "react";

type Todo = { id: number; text: string };

export default function TodoList() {
  const [todos, setTodos] = useState<Todo[]>([]);
  const [input, setInput] = useState<string>("");

  function addTodo() {
    if (!input.trim()) return;
    setTodos((prev) => [...prev, { id: Date.now(), text: input.trim() }]);
    setInput("");
```

```
  }

  function removeTodo(id: number) {
    setTodos((prev) => prev.filter((t) => t.id !== id));
  }

  return (
    <div>
      <input value={input} onChange={(e) => setInput(e.target.value)} />
      <button onClick={addTodo}>Add</button>
      <ul>
        {todos.map((t) => (
          <li key={t.id}>
            {t.text} <button onClick={() => removeTodo(t.id)}>✕</button>
          </li>
        ))}
      </ul>
    </div>
  );
}
```

> Both updates use the functional form ( `prev => ...` ) and produce **new arrays** with spread and `filter` — never `push` or `splice` .

## Data Flow

```
User interaction
      |
      ▼
  setState(next)
      |
      ▼
 React schedules a re-render
      |
      ▼
 Component runs again — useState returns the new value
      |
      ▼
 DOM updates
```

## Rules of `useState`

1. **Only call at the top of the component** — never inside `if`, loops, or nested functions. Hook order must be stable.
2. **Treat state as immutable** — always produce a new object/array; never mutate.
3. **Use the functional updater when the next value depends on the previous** — `setX((prev) => …)`.
4. **Don't store what you can derive** — compute during render instead of adding another `useState`.
5. **Wrap expensive initial values in a function** — `useState(() => compute())` runs only on the first render.
6. **Setter is stable** — its identity never changes, so it doesn't need to appear in `useEffect` deps.

---

## Common Bugs

### Bug 1 — Mutating state directly

```
const [items, setItems] = useState<number[]>([]);
items.push(1);          // ❌ mutation
setItems(items);        // same reference → React may skip re-render
```

**Fix:** `setItems((prev) => [...prev, 1]);`

### Bug 2 — Stale value in a queued update

```
setCount(count + 1);
setCount(count + 1);
setCount(count + 1);    // ❌ all three see the same old count → final result is +1
```

**Fix:** functional form, so each call reads the pending value.

```
setCount((c) => c + 1);
setCount((c) => c + 1);
setCount((c) => c + 1); // ✅ final result is +3
```

### Bug 3 — Reading state right after setting it

```
setCount(count + 1);
console.log(count);     // ❌ still the old value
```

`count` is a constant inside this render. The new value only exists in the **next** render. **Fix:** use a local variable, or react to the change in a `useEffect`.

### Bug 4 — Storing derived data in state

```
const [items, setItems] = useState<number[]>([]);
const [count, setCount] = useState<number>(0);   // ❌ can be derived

useEffect(() => { setCount(items.length); }, [items]);
```

**Fix:** compute in render.

```
const count = items.length;
```

### Bug 5 — Calling a hook conditionally

```
if (loggedIn) {
  const [name, setName] = useState("");   // ❌ Rules of Hooks violation
}
```

React tracks hooks by call order. Skipping one breaks all following hooks. **Fix:** call the hook unconditionally at the top of the component.

---

## `useState` vs `useReducer` — Code Comparison

**Counter with `useState`:**

```
const [count, setCount] = useState(0);

<button onClick={() => setCount((c) => c + 1)}>+</button>
```

**Same counter with `useReducer`:**

```
const [state, dispatch] = useReducer(reducer, { count: 0 });

<button onClick={() => dispatch({ type: "increment" })}>+</button>
```

> `useState` keeps things simple when the update logic fits in one line. Reach for `useReducer` once several actions share the same state.

## Quick Reference

| Scenario | Initial value | Update pattern |
|---|---|---|
| Counter / toggle | `0` / `false` | `setX((prev) => next)` |
| Text input | `""` | `setX(e.target.value)` |
| Enum / union | one of the union values | `setX(value as Category)` |
| List of items | `[]` | `[...prev, item]` / `prev.filter(...)` |
| Object with several keys | `{}` or `{...}` | `{ ...prev, key: value }` |
| Expensive initial value | `() => compute()` | as usual |
| Derived value | **not state** | compute during render |